CS2300+CS2310 — Foundations of Computer Systems Design (+ lab)

Tier	A
Phase (order)	3
Do this first	CS1111, CS1200
Estimated effort	12 weeks, ~8 h/week
Unlocks	CS2600, CS3500, CS3300

Roadmap

1. Boolean algebra and combinational logic (encoder, mux, adders, ALU).
2. Sequential logic: flip-flops, registers, counters, memory.
3. Build the whole computer bottom-up: CPU (Hack), assembler, VM, compiler, OS, as the course does.
4. Do the lab in Verilog/SystemVerilog and finish Tetris on your own machine.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Foundations to Computer Systems Design (77 videos). NPTEL *Foundations to Computer Systems Design* (V. Kamakoti, IIT Madras): this IS the IITM course.

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- Electronics - Digital Circuits and Systems (40 videos). NPTEL *Digital Circuits and Systems* (nptelhrd): the logic-design half in more depth.
- NOC Jan 2019: Digital Electronic Circuits (61 videos). NPTEL *Digital Electronic Circuits* (IIT Kharagpur, 2019).

⚡ Short videos for intuition and revision

- Building an 8-bit breadboard computer! (44 videos). Ben Eater *Building an 8-bit breadboard computer*: you will finally *see* how a CPU works.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–6	CMOS, gates, HDL simulator. Do nand2tetris Project 1.

Week	Lectures	What to do
2	7–13	Number systems, adders, the HACK ALU. Project 2.
3	14–22	Sequential logic, memory, PC, memory-mapped I/O. Projects 3 and 4 (machine language).
4	23–28	HACK CPU micro-architecture and the full computer. Project 5: build the CPU.
5	29–34	Assembler. Project 6: write the assembler in Python/C.
6-7	35–47	Virtual machine I and II. Projects 7-8: VM translator.
8-9	48–65	Jack language + compiler front end. Projects 9-10.
10-12	66–77	Compiler back end + OS. Projects 11-12. You now built a computer from NAND to a compiled program.

▼ **All lectures in *Foundations to Computer Systems Design* (77 videos)**

1. Module 1.1: Introduction to the Course
2. Module 1.2: CMOS Transistors and Gates
3. Module 1.3: Basic Gates
4. Module 1.4: Building Gates Using Simulator
5. Module 1.5: Hierarchical Design and Verification
6. Module 1.6: Building Blocks of a Digital Computer
7. Module 2.1: Binary Number Systems
8. Module 2.2: Signed Number Systems
9. Module 2.3: Two's Complement Number System
10. Module 2.4: Binary Adder Circuits
11. Module 2.5: Building the ALU of HACK
12. Module 2.6: HACK ALU Functionality
13. Module P1: Tips for Project P01
14. Module 3.1: Sequential Logic Design
15. Module 3.2: Latches and Flipflops
16. Module 3.3: The Memory Hierarchy
17. Module 3.4: Design of Program Counter
18. Module 3.5: Introduction to Computer Organization: The HACK Instruction Set Architecture (ISA)
19. Module 3.6 Memory Mapped I/O
20. Module P2: Tips for Projects P02 and P03
21. Module P3: Tips for Project 04
22. Module P4: Tips for Project 04
23. Module 4.1: Introduction to Computer Architecture
24. Module 4.2: The HACK Microarchitecture
25. Module 4.3: The HACK CPU - A Deep Dive: Part 1
26. Module 4.4: The HACK CPU - A Deep Dive: Part 2

27. Module 4.5: The Data Memory
28. Module 4.6: The HACK Computer
29. Module 5.1: The Assembler Construction
30. Module 5.2: Understanding the Working of Assembler
31. Module 5.3: Assembler: Symbol Table Construction
32. Module 5.4: Assembler: Pass 1
33. Module 5.5: Assembler: Pass 2
34. Module 5.6: Project 06 - Demonstration
35. Module 6.1: Virtual Machines - What and Why?
36. Module 6.2: The VM Instruction Set Architecture
37. Module 6.3: The execution of a VM Program
38. Module 6.4: How powerful is the VM?
39. Module P5: Project 07: VM ISA to HACK Mnemonic Translation
40. Module 6.6: Project 07: Demo
41. Module 6.5: Deep Understanding of VM ISA using VM Emulator
42. Module 7.1: Virtual Machine II - Program flow commands and Introduction to Function Calls
43. Module 7.2: Implementation of Function Call
44. Module 7.3: Working of the Virtual Machine
45. Module 7.4: Project 08: Translation of Program Flow and Function Call to HACK Mnemonic
46. Module 7.5: Handling Static Variables
47. Module 7.6: Project 08: Implementation tips in a Nut Shell
48. Module 8.1: Introduction to The JACK Programming Language
49. Module 8.2: Project 09: Basic Steps
50. Module 8.3: Understanding Syntax of JACK using Examples
51. Module 8.4: Project 09: More Examples
52. Module 8.5: The JACK Syntax - Language Specification
53. Module 8.6: Application Development using JACK
54. Module 9.1: JACK Compiler: Lexical Analysis - Tokenization
55. Module 9.2: Project 10: Compiler for JACK - Part-1 Demo
56. Module 9.3: The JACK Grammar
57. Module 9.4: Compiler for JACK: Parsing the JACK Program
58. Module 9.5: Project 10: Part 2: The Token Analyzer
59. Module 9.6: Project 10: Part 3: Testing the Correctness
60. Mod 10 1
61. Mod 10 2
62. Mod 10 3
63. Mod 10 4
64. Mod 10 5
65. Mod 10 6
66. Module 11.1: The Jack Compiler Backend: An XML guided approach
67. Module 11.2: The Jack Compiler Backend: Populating the Class and Subroutine Symbol tables.

68. Module 11.3: The Jack Compiler Backend: Code Generation - 1
69. Module 11.4: The Jack Compiler Backend: Code Generation - 2
70. Module 11.5: The Jack Compiler Backend: Code Generation - 3
71. Module 11.6: The Jack Compiler Backend: Code Generation - 4
72. Module 12.1: Jack Compiler: Code Generation - 5
73. Module 12.2: Jack Compiler: Code Generation - 6
74. Module 12.3: Jack Compiler: Code Generation - 7
75. Module 12.4: Understand the Operating System - Compiler Interactions
76. Module 12.5: Project - 12: One sample journey from Jack to Hack
77. Module 12.6: Concluding Remarks

Books, notes and websites

- ✅ NPTEL *Foundations to Computer Systems Design* (Kamakoti, IIT Madras) — onlinecourses.nptel.ac.in/noc20_cs34/preview
- nand2tetris.org — the official reference text is *The Elements of Computing Systems*; free Coursera Parts I and II
- HDLBits (hdlbits.01xz.net) for Verilog drills
- Book: *Code* (Charles Petzold) for intuition

Practice / proof of understanding

Complete nand2tetris projects 1-12 (gates -> ALU -> CPU -> assembler -> VM -> compiler -> OS).

What your IITM course sheet says

From the PDF: CS2300

Official description: This course is intended to understand the different components of a computer system, namely, digital hardware, the microarchitecture, the operating systems and the compilers, and their interconnection.

Official syllabus (trimmed):

1. Introduction: Functional components of a computer system, Interaction among functional components
2. Design of Arithmetic and Logic Unit: Digital logic gates, Boolean algebra, Simplification of Boolean Expressions, Combinational logic circuits – Encoder, Decoder, Multiplexer and Demultiplexer; Representation of integer data, Integer adders, Integer multipliers, Design of integer unit, Floating point representation of real data, Floating-point adder/subtractor, Floating-point multiplier, Design of Floating point unit
3. Design of Register File and Memory Unit: Flip-flops, Synchronous sequential circuits – Registers and Counters; Memory unit
4. Generation and Execution of Machine Code: Instruction set architecture of a simple CPU, Assembler, Code generation, Application binary interface, Microarchitecture of CPU, Execution of machine code
5. Processing of High Level Language Code: Virtual machine, Interpreter, A simple compiler, Fundamentals of operating system

Official textbooks: 1. C. H. Roth and L.L.Kinney, Fundamentals of Logic Design, 7th Edition, Cengage Learning, 2014. 2. C.Hamacher, Z.Vranesic, S.Zaky and N.Manjikian, Computer Organization and Embedded Systems, 6th Edition, McGraw-Hill, 2012 3. N. Nisan and S. Schocken, The Elements of Computing Systems – Building a Modern Computer from First Principles, The MIT Press, 2005

Official references: 1. V. P. Nelson, H.T. Nagle, E.D. Caroll and J.D. Irwin, Digital Logic Circuit Analysis and Design, Prentice Hall International, 1995 2. S. Brown and Z Vranesic, Fundamentals of Logic Design with VHDL Design, Tata McGraw-Hill , 2000 4. F.J. Hill and G.R.Peterson, Computer Aided Logical Design with Emphasis on VLSI, John Wiley & Sons, 1993

From the PDF: CS2310

Official description: This is the laboratory course, associated with CS2300 - Foundations of Computer Systems Design. In the lab part – the components taught in the course shall be described using a suitable description language and integrated to form a system; the assembler, compiler and OS concepts shall be ported on the system built above.

Official syllabus (trimmed):

1. Breadboard exercises a. 4-to-2 bit Encoder b. 1-bit, 4-to-1 Line Multiplexer c. 4-bit ripple carry adder
2. Hardware Description Language (such as Verilog) based exercises: a. 16-bit Adder/Subtractor using 4-bit Carry Look Ahead circuits b. 16-bit Signed integer multiplier using Wallace tree multiplier c. Arithmetic and Logic unit for 16-bit operands d. Register file and Main memory
3. Implementation of microarchitecture a simple processor (such as Hack)
4. Interpreter for the language of a virtual machine (such as JACK)
5. Compiler for a simple language (such as JACK)
6. Compilation and execution of a complete application (such as Tetris game)

Official textbooks: S. Sutherland, S. Davidmann and P. Flake, SystemVerilog for Design - A Guide to Using System Verilog for Hardware Design and Modeling, 2nd Edition, Springer, 2006.

Official references: N. Nisan and S. Schocken, The Elements of Computing Systems – Building a Modern Computer from First Principles, The MIT Press, 2005